TÀI LIỆU HỌC LẬP TRÌNH

React · Node.js · Git

Subject: FER202
Name: Nguyễn Minh Trung
ID Student: DE190522

Contents

1. React là gì? 1
Lợi ích của React 1
2. Các thành phần của React 1
2.1. Component (Thành phần) 1
2.2. JSX (JavaScript XML) 2
5. Git là gì? 2
5.1. Giới thiệu 2
5.2. Khái niệm cơ bản 2
5.3. Các lệnh Git cơ bản 3
5.4. Quy trình làm việc thông thường 4
2.3. Props (Thuộc tính) 4
2.4. State (Trạng thái) 5
2.5. Hooks 5
3. Virtual DOM là gì? 6
3.1. DOM thật (Real DOM) là gì? 6
3.2. Virtual DOM (DOM ảo) hoạt động như thế nào? 6
3.3. Ví dụ minh họa cụ thể 6
4. Node.js và NPM là gì? 7
4.1. Node.js là gì? 7
4.2. NPM (Node Package Manager) là gì? 8

1. React là gì?

React là một **thư viện JavaScript** mã nguồn mở do Facebook phát triển (ra mắt năm 2013), dùng để xây dựng giao diện người dùng (UI) cho các trang web.

Hãy tưởng tượng: Khi bạn dùng Facebook, mỗi khi like bài viết, số lượt like tăng lên ngay mà không cần tải lại trang. Đó chính là React đang hoạt động!

Lợi ích của React

- **Nhanh & mượt mà:** React chỉ cập nhật phần thay đổi trên trang, không tải lại toàn bộ.
- **Tái sử dụng code:** Viết một component, dùng ở nhiều nơi trong ứng dụng.
- **Cộng đồng lớn:** Hàng triệu lập trình viên, dễ tìm tài liệu và hỗ trợ.
- **Dễ học:** Cú pháp gần giống HTML nên người mới dễ tiếp cận.
- **Linh hoạt:** Kết hợp tốt với nhiều thư viện và công cụ khác.

2. Các thành phần của React

2.1. Component (Thành phần)

Component là **khối xây dựng cơ bản** của React. Hiểu đơn giản: một trang web được ghép lại từ nhiều component nhỏ, giống như LEGO.

Ví dụ: Trang Facebook có các component: Header, Sidebar, NewsFeed, Post, LikeButton, ...

Ví dụ code component đơn giản:

```
// Đây là một Component tên là 'ChaoHoi'
function ChaoHoi() {
  // return: trả về giao diện cần hiển thị
  return (
    <div>
      <h1>Xin chào! Tôi là React!</h1>   {/* Tiêu đề */}
      <p>Rất vui được gặp bạn.</p>       {/* Đoạn văn */}
    </div>
  );
}


// Sử dụng component (như thẻ HTML tự tạo)
```

```
<ChaoHoi />   // --> Hiển thị: Xin chào! Tôi là React!
```

2.2. JSX (JavaScript XML)

JSX là cú pháp đặc biệt cho phép viết **HTML bên trong JavaScript**. Trình duyệt không hiểu JSX trực tiếp, React sẽ chuyển đổi nó thành JavaScript.

```
// JSX - viết giống HTML trong JavaScript
const tieuDe = <h1>Xin chào!</h1>;
//             ^^^^^^^^^^^^^^^^^ Đây là JSX


// Có thể nhúng biến JavaScript vào JSX bằng {}
const ten = 'Minh';
const loi_chao = <h1>Xin chào, {ten}!</h1>;
// Kết quả hiển thị: 'Xin chào, Minh!'
```

5. Git là gì?

5.1. Giới thiệu

Git là **hệ thống quản lý phiên bản** (Version Control System). Nó ghi lại mọi thay đổi trong code của bạn theo thời gian.

> 💡 **Ví dụ dễ hiểu:** Git giống như nút 'Ctrl+Z' siêu mạnh và thông minh. Bạn có thể quay lại bất kỳ phiên bản nào trong quá khứ, làm việc nhóm mà không đè code của nhau!

Git giải quyết vấn đề gì?

- **Không sợ mất code:** Mọi thay đổi đều được lưu lại
- **Làm việc nhóm:** Nhiều người cùng code một dự án mà không xung đột
- **Thử nghiệm an toàn:** Tạo nhánh (branch) mới để thử tính năng, không ảnh hưởng code gốc
- **Xem lịch sử:** Biết ai thay đổi gì, khi nào

5.2. Khái niệm cơ bản

Repository (Repo): Thư mục dự án được Git quản lý. Có hai loại:

- **Local repo:** Trên máy tính của bạn
- **Remote repo:** Trên server online (GitHub, GitLab...)

Commit: Một 'checkpoint' lưu trạng thái code tại một thời điểm, kèm theo mô tả.

Branch (Nhánh): Phiên bản song song của dự án. Ví dụ: nhánh main là code chính, nhánh feature/login là code tính năng đăng nhập đang phát triển.

5.3. Các lệnh Git cơ bản

Nhóm 1: Khởi tạo & Cấu hình

```
# Thiết lập tên và email (làm 1 lần duy nhất)
git config --global user.name 'Nguyen Van A'
git config --global user.email 'a@email.com'

# Khởi tạo Git cho thư mục hiện tại
git init
# Kết quả: tạo thư mục .git để Git theo dõi

# Sao chép repo từ GitHub về máy
git clone https://github.com/ten/du-an.git
# Kết quả: tải toàn bộ code và lịch sử về máy
```

Nhóm 2: Làm việc hằng ngày

```
# Xem trạng thái: file nào thay đổi, file nào chưa lưu
git status

# Thêm file vào 'staging area' (chuẩn bị commit)
git add ten-file.js        # Thêm 1 file cụ thể
git add .                  # Thêm TẤT CẢ file thay đổi

# Lưu checkpoint với mô tả
git commit -m 'Thêm tính năng đăng nhập'
#            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Mô tả ngắn, rõ ràng

# Xem lịch sử commit
git log                    # Chi tiết đầy đủ
git log --oneline          # Tóm tắt 1 dòng mỗi commit
```

Nhóm 3: Branch (Nhánh)

```
# Xem danh sách nhánh (* là nhánh đang dùng)
git branch

# Tạo nhánh mới
git branch tinh-nang-gio-hang
```

```
# Chuyển sang nhánh khác
git checkout tinh-nang-gio-hang

# Tạo nhánh MỚI và chuyển sang luôn (lệnh tắt)
git checkout -b tinh-nang-thanh-toan

# Gộp nhánh vào nhánh hiện tại
git merge tinh-nang-gio-hang
```

Nhóm 4: Làm việc với Remote (GitHub)

```
# Đẩy code lên GitHub
git push origin main        # Đẩy nhánh 'main' lên

# Kéo code mới nhất từ GitHub về
git pull origin main        # Kéo và gộp vào nhánh hiện tại

# Xem danh sách remote (server) đã kết nối
git remote -v

# Kết nối repo local với repo trên GitHub
git remote add origin https://github.com/ten/du-an.git
```

5.4. Quy trình làm việc thông thường

```
# Bước 1: Kéo code mới nhất về trước khi bắt đầu
git pull origin main

# Bước 2: Tạo nhánh mới cho tính năng đang làm
git checkout -b tinh-nang-moi

# Bước 3: Viết code... xong rồi lưu lại
git add .
git commit -m 'Mô tả những gì bạn vừa làm'

# Bước 4: Đẩy nhánh lên GitHub
git push origin tinh-nang-moi

# Bước 5: Tạo Pull Request trên GitHub để review & gộp code
```

2.3. Props (Thuộc tính)

Props là cách truyền **dữ liệu từ component cha sang component con**. Giống như bạn gọi điện thoại và nhập số điện thoại - số đó là 'props'.

```
// Component nhận props là 'ten' và 'tuoi'
function GioiThieu(props) {
  return (
    <p>
      Tôi tên {props.ten},       {/* Lấy giá trị 'ten' từ props */}
      năm nay {props.tuoi} tuổi. {/* Lấy giá trị 'tuoi' từ props */}
    </p>
  );
}

// Sử dụng: truyền dữ liệu vào component
<GioiThieu ten='An' tuoi={22} />
// Kết quả: 'Tôi tên An, năm nay 22 tuổi.'
```

2.4. State (Trạng thái)

State là **dữ liệu nội bộ** của component, có thể thay đổi. Khi state thay đổi, React tự động cập nhật giao diện.

```
import { useState } from 'react'; // Import hook useState

function DemSo() {
  // Tạo state: 'dem' là giá trị, 'setDem' là hàm để thay đổi
  // useState(0) --> giá trị ban đầu là 0
  const [dem, setDem] = useState(0);

  return (
    <div>
      <p>Bạn đã bấm: {dem} lần</p>

      {/* Khi bấm nút, gọi setDem để tăng dem lên 1 */}
      <button onClick={() => setDem(dem + 1)}>
        Bấm vào đây!
      </button>
    </div>
  );
}
// Mỗi lần bấm nút, số đếm tự động tăng lên!
```

2.5. Hooks

Hooks là các hàm đặc biệt giúp component sử dụng các tính năng nâng cao của React. Một số Hooks thông dụng:

- **useState:** Quản lý dữ liệu có thể thay đổi (như ví dụ trên)

- **useEffect:** Thực hiện hành động khi component tải xong (gọi API, lấy dữ liệu...)
- **useContext:** Chia sẻ dữ liệu giữa nhiều component mà không cần truyền qua props

3. Virtual DOM là gì?

3.1. DOM thật (Real DOM) là gì?

DOM (Document Object Model) là **cây cấu trúc HTML** mà trình duyệt tạo ra khi tải trang web. Mỗi thẻ HTML là một 'nút' trong cây này.

```
<!-- HTML của bạn -->          DOM thật (cây cấu trúc):
<html>                      html
  <body>                     └── body
    <div id='app'>                └── div#app
      <h1>Tiêu đề</h1>                ├── h1 ('Tiêu đề')
      <p>Nội dung</p>                 └── p ('Nội dung')
    </div>
  </body>
</html>
```

⚠ **Vấn đề:** Cập nhật DOM thật rất chậm! Mỗi lần thay đổi, trình duyệt phải vẽ lại toàn bộ trang.

3.2. Virtual DOM (DOM ảo) hoạt động như thế nào?

Virtual DOM là một **bản sao nhẹ của DOM thật** được React lưu trong bộ nhớ. React so sánh Virtual DOM cũ và mới, chỉ cập nhật phần thực sự thay đổi.

Quy trình 3 bước của React:

1. **Render:** React tạo Virtual DOM mới khi có thay đổi
2. **Diff (So sánh):** React so sánh Virtual DOM cũ và mới (thuật toán 'diffing')
3. **Patch (Vá):** React chỉ cập nhật đúng phần thay đổi lên DOM thật

3.3. Ví dụ minh họa cụ thể

Tình huống: Trang web có danh sách 100 sản phẩm. Bạn cập nhật giá của 1 sản phẩm.

❌ Không có Virtual DOM	☑ Có Virtual DOM (React)
1. Thay đổi giá sản phẩm #47	1. Thay đổi giá sản phẩm #47
2. Xóa và vẽ lại TẤT CẢ 100 sản phẩm	2. So sánh và tìm ra: chỉ #47 thay đổi
3. Chậm, tốn tài nguyên!	3. Chỉ cập nhật đúng ô giá #47
	4. Nhanh, tiết kiệm tài nguyên!

Ví dụ code:

```
// Danh sách sản phẩm ban đầu
const [sanPham, setSanPham] = useState([
  { id: 1, ten: 'Táo',  gia: 10000 },
  { id: 2, ten: 'Cam',  gia: 15000 },
  { id: 3, ten: 'Xoài', gia: 20000 },
]);

// Cập nhật giá Cam (id=2) thành 18000
// React chỉ cập nhật dòng 'Cam' trên màn hình, không vẽ lại toàn bộ!
setSanPham(sanPham.map(sp =>
  sp.id === 2                    // Tìm sản phẩm có id = 2
    ? { ...sp, gia: 18000 }        // Nếu đúng: đổi giá thành 18000
    : sp                         // Nếu sai: giữ nguyên
));
```

4. Node.js và NPM là gì?

4.1. Node.js là gì?

Node.js là môi trường cho phép chạy **JavaScript bên ngoài trình duyệt**, tức là trực tiếp trên máy tính (server).

> 💡 **Ví dụ dễ hiểu:** JavaScript thông thường chỉ chạy trong Chrome/Firefox. Node.js giống như mang JavaScript ra ngoài trình duyệt, để bạn dùng nó viết server, đọc file, tạo app dòng lệnh...

Node.js dùng để làm gì?

- **Xây dựng server/API:** Nhận yêu cầu từ trình duyệt, trả về dữ liệu
- **Đọc/ghi file:** Thao tác với file trên máy tính
- **Chạy công cụ dev:** React, Vue, TypeScript đều cần Node.js để hoạt động
- **Ứng dụng real-time:** Chat, thông báo trực tiếp, ...

```
// server.js - Server Node.js đơn giản nhất
const http = require('http'); // Import thư viện http có sẵn của Node.js

// Tạo server
const server = http.createServer((req, res) => {
  // req: thông tin yêu cầu từ trình duyệt
  // res: phản hồi gửi về cho trình duyệt
  res.end('Xin chào! Đây là server Node.js!');
});

// Lắng nghe ở cổng 3000
server.listen(3000, () => {
  console.log('Server đang chạy tại http://localhost:3000');
});
```

4.2. NPM (Node Package Manager) là gì?

NPM là **trình quản lý gói (package)** đi kèm với Node.js. 'Gói' là các thư viện/code do người khác viết sẵn mà bạn có thể tải về và dùng miễn phí.

🛒 **Ví dụ dễ hiểu:** NPM giống như App Store/Google Play, nhưng dành cho lập trình viên. Bạn muốn thêm tính năng mới? Tìm và cài gói có sẵn thay vì tự viết từ đầu!

Các lệnh NPM thường dùng:

```
# Khởi tạo dự án mới (tạo file package.json)
npm init

# Cài đặt một gói (ví dụ: React)
npm install react
# Viết tắt: npm i react

# Cài tất cả gói trong file package.json (khi clone project về)
npm install

# Gỡ cài đặt một gói
npm uninstall react

# Xem danh sách gói đã cài
npm list

# Tạo ứng dụng React mới (lệnh phổ biến nhất khi bắt đầu)
npx create-react-app ten-du-an
```